

VET — Vulnerable Exploitation Tool : Un Assistant IA pour l'Analyse et la Priorisation des Vulnérabilités en SOC

Masha Mirza Zad Ziabary, Abdullah Ahmed-Bark Swailem,
Mirwis Satarzai, and Damien Maris

Master 2 Cybersécurité — Projet VET

Résumé Ce rapport présente le projet VET (Vulnerable Exploitation Tool), un assistant intelligent conçu pour aider les analystes SOC (Security Operations Center) à prioriser et gérer les vulnérabilités. Le système combine un pipeline RAG (Retrieval-Augmented Generation) basé sur LangChain et le modèle Llama 3.1 8B, un moteur de priorisation hybride associant un arbre de décision VMC (Vulnerability Management Chaining) et un modèle de Machine Learning XGBoost, ainsi qu'un analyseur de logs de sécurité temps réel. L'outil s'appuie sur une base de données SQLite enrichie depuis NVD, CISA KEV et EPSS, indexée dans un vectorstore FAISS pour la recherche sémantique. Une infrastructure de serveur vulnérable simulé sous Docker permet de valider l'outil en conditions réalistes.

Keywords: SOC · RAG · LLM · Vulnérabilités · XGBoost · CVSS · EPSS · KEV · LangChain

1 Introduction

1.1 Contexte et problématique

Dans un centre opérationnel de sécurité (SOC), les analystes sont confrontés quotidiennement à un volume considérable de vulnérabilités détectées sur les infrastructures. Le nombre de CVE (Common Vulnerabilities and Exposures) publiées chaque année ne cesse de croître, dépassant les 28 000 en 2023. Face à ce déluge d'informations, une question centrale se pose : **comment un assistant SOC peut-il aider à faire la distinction entre les vulnérabilités réellement critiques et les vulnérabilités théoriques ?**

Les scores CVSS (Common Vulnerability Scoring System) [4], bien qu'indispensables, ne suffisent pas à eux seuls pour capturer la réalité du risque. Une vulnérabilité notée 9.8 en CVSS peut être théorique si aucun exploit n'existe, tandis qu'une vulnérabilité notée 7.0 activement exploitée (référéncée dans le catalogue KEV [3]) représente un danger immédiat.

1.2 Objectif du projet

Le projet VET (Vulnerable Exploitation Tool) propose un assistant IA qui combine :

- Un **modèle de langage** (LLM) local pour l'analyse contextuelle en langage naturel,
- Un **système RAG** (Retrieval-Augmented Generation) [10] pour enrichir les réponses avec des données factuelles,
- Un **moteur de priorisation hybride** combinant règles déterministes (VMC) et Machine Learning (XGBoost) [2],
- Un **analyseur de logs** pour corréler les vulnérabilités avec les traces d'exploitation réelles.

L'objectif est de fournir à l'analyste SOC un outil conversationnel intelligent capable de répondre à des questions complexes telles que : « *Quelles sont les vulnérabilités les plus critiques sur mon serveur ?* », « *La CVE-2021-44228 est-elle exploitée dans nos logs ?* », ou encore « *Que recommandes-tu comme plan d'action ?* ».

1.3 Organisation du rapport

Ce rapport est organisé comme suit. La section 2 présente l’architecture globale du système. La section 3 détaille l’infrastructure de simulation et la génération de données. La section 4 décrit le pipeline d’ingestion des données. La section 5 présente le moteur de priorisation VMC et XGBoost. La section 6 explique le système RAG et l’agent LLM. La section 7 aborde les travaux connexes. Enfin, la section 8 conclut et propose des perspectives d’amélioration.

2 Architecture globale du système

2.1 Vue d’ensemble

VET (Vulnerability Evaluation Tool) est conçu comme un pipeline modulaire multi-couches visant à automatiser l’analyse de sécurité, la priorisation des vulnérabilités et la génération de rapports SOC assistés par des modèles d’intelligence artificielle. L’architecture repose sur un paradigme *agent-outils*, dans lequel un agent central intelligent, nommé **SecurityAssistant**, orchestre dynamiquement l’exécution de composants spécialisés en fonction de l’intention détectée dans la requête utilisateur.

Comme illustré à la Figure 1, le système est structuré en plusieurs couches fonctionnelles clairement séparées afin d’assurer la scalabilité, la maintenabilité et l’évolutivité de la plateforme.

Couche présentation L’interaction utilisateur est assurée par une application web interactive développée avec **Streamlit**. Cette interface permet aux analystes SOC de soumettre des requêtes en langage naturel, de visualiser les résultats d’analyse, ainsi que de consulter des rapports de sécurité générés automatiquement. L’application est accessible via un navigateur web (Chrome/Firefox) et communique avec le backend via HTTPS.

Couche orchestration Au cœur du système se trouve le composant **SecurityAssistant**, implémenté comme un agent intelligent capable de raisonner sur la requête utilisateur. Il s’appuie sur un module de classification d’intentions afin d’identifier le type d’analyse demandé (analyse de logs, évaluation de vulnérabilités, priorisation des risques, génération de rapport, etc.). En fonction de cette intention, l’agent sélectionne et invoque les outils appropriés tout en maintenant un flux de contrôle cohérent.

Couche moteurs Cette couche regroupe les moteurs d’analyse principaux :

- le **Data Engine**, chargé de l’ingestion, de la normalisation et du scoring initial des données issues de différentes sources (XML, JSON, logs bruts) ;
- le **Detection Engine**, responsable de l’analyse des journaux système et applicatifs afin de détecter des événements de sécurité suspects ;
- le **Priority Engine**, qui applique des règles métiers et des modèles prédictifs pour arbitrer et prioriser les vulnérabilités selon leur criticité.

Couche outils spécialisés Plusieurs outils indépendants sont exposés à l’agent central :

- un calculateur de risque combinant les scores EPSS et l’analyse des logs ;
- un moteur de recommandation SOC basé sur des modèles de langage ;
- un outil de recherche hybride (*retriever*) combinant SQL et recherche vectorielle pour l’exploration des bases de vulnérabilités ;
- un module de scoring VMC ;
- un générateur de rapports SOC au format PDF.

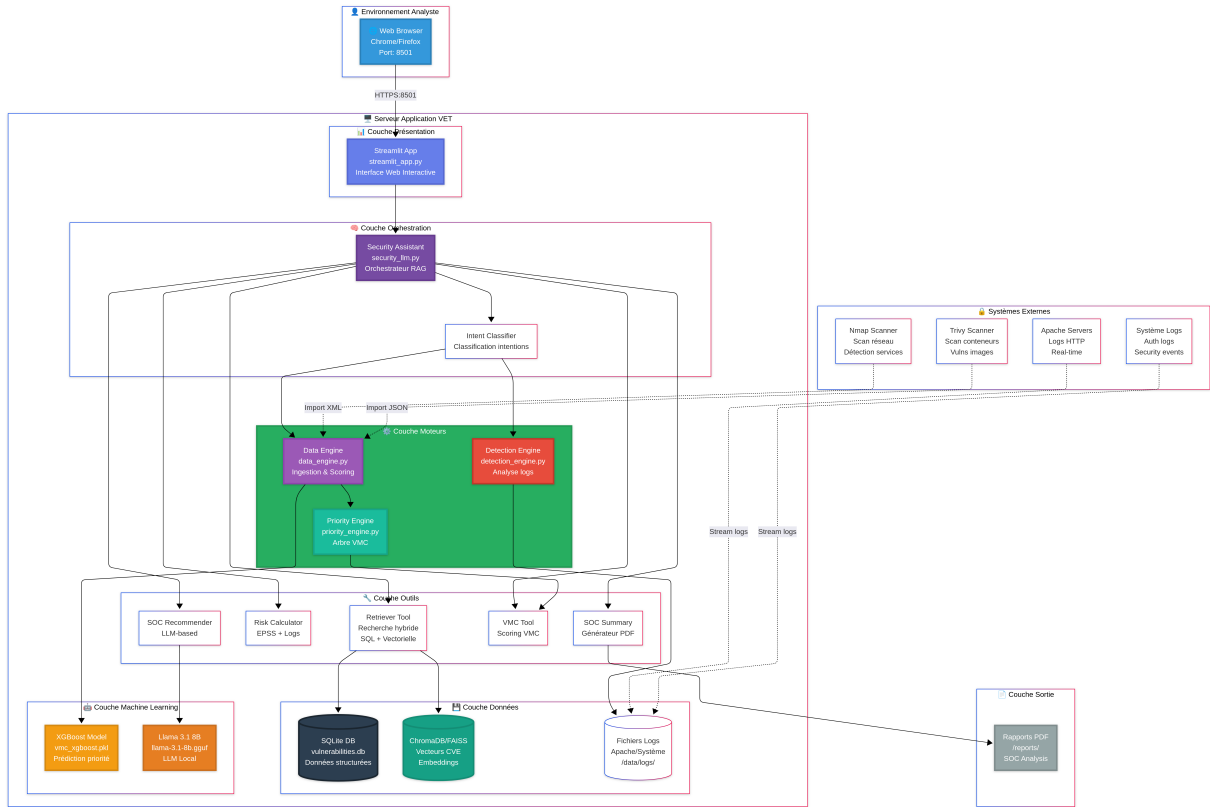


FIGURE 1. Architecture globale du système VET.

Couche données et apprentissage Les données sont stockées dans des bases structurées (SQLite) et vectorielles (ChromaDB/FAISS) afin de permettre à la fois des requêtes classiques et des recherches sémantiques. La couche d'apprentissage automatique intègre des modèles locaux, notamment un modèle XGBoost pour la prédiction de priorité et un LLM (LLaMA 3.1) utilisé pour l'assistance contextuelle et la génération de synthèse

2.2 Structure des fichiers

Le projet est organisé selon la structure suivante :

Fichier/Dossier	Rôle
<code>security_llm.py</code>	Agent RAG principal (orchestrateur LangChain)
<code>data_engine.py</code>	Ingestion de données et moteur de scoring VMC/ML
<code>priority_engine.py</code>	Logique de priorisation (arbre de décision VMC)
<code>detection_engine.py</code>	Moteur de détection de traces d'exploitation
<code>config.py</code>	Configuration centralisée (LLM, ML, DB, chemins)
<code>streamlit_app.py</code>	Interface Web (Streamlit)
<code>tools/</code>	Outils de l'agent (retriever, VMC, risk, recommander, summary)
<code>utils/</code>	Modules utilitaires (vectorstore, log_analyzer)
<code>data/</code>	Données (SQLite, ChromaDB, logs, loaders)
<code>prompts/</code>	Templates de prompts centralisés
<code>models/</code>	Modèles ML (XGBoost) et LLM (GGUF)

TABLE 1. Structure des fichiers du projet VET.

2.3 Pipeline de traitement

Le flux de traitement d'une question utilisateur suit les étapes suivantes :

1. **Réception** : L'utilisateur pose une question via l'interface Streamlit [18].
2. **Classification d'intention** : Le LLM classe la question parmi 8 intentions (`list_vulns`, `analyze_cve`, `recommend`, `logs`, `search`, `summary`, `compromise`, `general_security`).
3. **Activation des outils** : Selon l'intention, les outils pertinents sont appelés pour collecter les données nécessaires.
4. **Construction du contexte RAG** : Les résultats des outils sont fusionnés en un contexte structuré dédoublé.
5. **Génération structurée (Pass 1)** : Le LLM génère une réponse structurée via des schémas Pydantic [14].
6. **Reformulation naturelle (Pass 2)** : Le LLM reformule la réponse structurée en dialogue naturel pour l'analyste.

3 Infrastructure de simulation et d'évaluation

Pour tester notre solution VET de manière réaliste, nous avons conçu une infrastructure de simulation complète. Celle-ci repose sur un serveur volontairement vulnérable, *FileTransferLike*, qui nous donne la possibilité de générer différents vecteurs d'attaques et de voir si notre IA parvient à les corrélés.

3.1 Architecture et configuration du serveur sous Docker

L'environnement utilise un conteneur Docker unique (`getmynote-server`) pour faciliter son déploiement et son orchestration. L'idée est de simuler un serveur d'entreprise accumulant de la dette technique, qui n'aurait pas été mis à jour depuis plusieurs années.

Réalisme et limites. Il est important de préciser que la nature même de ce conteneur constitue un biais volontaire : il s'agit d'un "pot de miel" de laboratoire particulièrement concentré. Les vulnérabilités qui y sont rassemblées sont extrêmement directes et nombreuses pour un seul système. Cela offre un scénario d'apprentissage idéal pour l'entraînement d'une IA de SOC, bien qu'il soit potentiellement trop dense et asymétrique pour représenter un serveur de production standard. Ce parti pris garantit en revanche un apport massif et immédiat de cas pratiques.

Les choix techniques définis dans notre fichier `Dockerfile` sont les suivants :

- **Système d’exploitation : Ubuntu 18.04.** Son support officiel étant terminé depuis avril 2023, cela nous garantit d’avoir des paquets obsolètes. Nous y avons par exemple forcé l’installation d’une ancienne version de l’interpréteur de commandes Bash pour pouvoir exploiter la fameuse faille Shellshock.
- **Application Web cible :** Une plateforme de transfert de fichiers développée en **Python/Flask** autour de 1300 lignes de code et servie par le module **Apache WSGI** sur le port 8000.
- **Services réseaux obsolètes :** Nous avons exposé plusieurs services vulnérables, comme **MySQL 5.7** avec un compte administrateur sans mot de passe, **Redis 5.0.3** accessible directement depuis l’extérieur sur l’adresse réseau par défaut, ainsi que **Samba SMBv1** pour exploiter EternalBlue et **VSFTPD 3.0.3** qui autorise les envois de fichiers anonymes et inclut une backdoor classique.
- **Mauvaises configurations système :** Nous avons compilé une ancienne version d’**OpenSSL** pour simuler Heartbleed. Le système comporte aussi des oublis graves d’administration, comme des droits en lecture et écriture élargis sur le fichier des mots de passe **/etc/shadow**, l’autorisation de se connecter en SSH en tant qu’administrateur sans mot de passe, ou encore l’usage de la commande **sudo** sans restriction. Cela facilite considérablement l’exploitation d’une faille locale d’élévation de privilège comme la CVE-2021-3156.

3.2 Choix, répartition et criticité des vulnérabilités

Au total, l’infrastructure implémente et documente **89 vulnérabilités**, dont 21 d’infrastructure et 68 applicatives. Ce panel n’est pas aléatoire : il sert à fournir à l’IA une base de référence solide, le ground truth, diversifiée pour couvrir tout le spectre d’une attaque, des failles de criticité faible comme les fuites d’informations aux failles de criticité maximale.

Nous avons réparti ces failles en trois grandes catégories :

1. **Faillles logiques Web :** On y retrouve les failles majeures du Top 10 OWASP comme les injections SQL, les accès illégitimes aux fichiers via IDOR, et les falsifications de requêtes SSRF. Ce type de failles a été intégré car il génère d’importants volumes de logs HTTP d’erreur sous Apache, fournissant à l’IA des motifs clairs d’injections à analyser dans le trafic entrant.
2. **Exécutions de code et Injections SSTI/RCE :** L’application inclut des injections de templates serveur SSTI, un point de terminaison de diagnostic mal filtré menant à des injections de commandes, ainsi que de la désérialisation Python vulnérable. L’exploitation de ces vecteurs, souvent classée comme critique avec un score CVSS supérieur à 9.0, permet d’aboutir à une exécution de code à distance complète RCE. Cela déclenche des alertes système sous Auditd et l’apparition de processus enfants illégitimes repérables dans la télémétrie.
3. **Faillles d’infrastructure et post-exploitation :** L’intégration de protocoles réseau en clair comme Telnet, de bases de données NoSQL non protégées et de failles locales Sudo est primordiale pour valider le modèle. Bien qu’elles soient souvent triviales à exploiter, elles nous permettent de simuler finement les étapes de mouvements latéraux et d’élévation de privilèges, des phases décisives de la kill chain qu’un SOC doit impérativement être capable d’intercepter.

3.3 Génération de trafic et collecte de télémétrie

Pour alimenter notre IA en données, nous avons automatisé le cycle de vie des attaques.

Un script d’activité simule le comportement du trafic légitime et malveillant. Il lance des requêtes HTTP ciblées, effectue des scans automatiques avec des outils d’audit comme **OWASP ZAP**, et réalise des attaques par force brute sur les ports SSH et FTP.

Pour centraliser cette télémétrie, notre conteneur récupère plus de 25 sources de logs différentes grâce aux agents **Filebeat** et **Auditbeat**, organisées en trois flux majeurs :

- **Logs Web et Base de données :** Logs d’accès Apache, traces applicatives de Gunicorn, et un fichier JSON sécuritaire développé en interne où l’application signale elle-même les tentatives d’injections ou comportements anormaux détectés au niveau du code.

- **Logs Système** : Capture des événements système pour surveiller le téléchargement de charges utiles inattendues, les succès et échecs d'authentification SSH, ainsi que les appels sudo inhabituels.
- **Logs Réseaux** : Enregistrement des exfiltrations ou des envois de webshells via le service FTP.

Un module Python s'occupe ensuite de parser et normaliser ces événements bruts avec la bibliothèque `pyGrok`. Tout est formaté en JSON, ce qui prépare le format d'ingestion adéquat pour notre modèle de détection de Machine Learning.

3.4 Évaluation par scanners, validation et analyse comparative

Afin d'obtenir une base de comparaison solide pour le projet, nous avons audité notre environnement de test avec trois scanners industriels de référence. Cette analyse croisée permet à la fois de garantir que notre architecture répond bien aux attaques automatisées classiques, et de confronter notre déploiement initial de vulnérabilités à ce qui ressort de scans professionnels effectifs.

Outil	Approche de scan	Visibilité sur l'infrastructure
OWASP ZAP	Scan Web Actif DAST	Cible l'application Flask et confirme de manière catégorique la présence des vulnérabilités web intégrées SQLi, CSRF, SSTI.
OpenVAS	Audit d'Infrastructure	Évalue la surface réseau, la robustesse des services exposés et identifie les faillibilités matérielles.
Trivy	Scan de conteneurs SCA	Liste les dépendances système et logicielles obsolètes liées à l'image Ubuntu 18.04 et examine le code source.

TABLE 2. Outils d'audit utilisés et périmètre couvert sur FileTransferLike.

Analyse des détections et concordances. La compilation de ces rapports génère plus de 175 alertes distinctes sur l'ensemble du périmètre. L'étude détaillée des occurrences démontre la fiabilité du protocole de simulation :

- **Succès de la Détection DAST avec OWASP ZAP** : ZAP remonte avec pertinence des failles logiques très attendues. L'analyse de son rapport identifie avec succès l'Injection SQL sur la page de connexion, le Server Side Template Injection intégré dans le moteur de rendus HTML, et détecte correctement l'absence de jetons Anti-CSRF voulue lors des suppressions de comptes, ainsi que de multiples redirections ouvertes. L'exposition flagrante des commentaires noyés dans notre code et l'absence radicale d'en-têtes de sécurité sont également explicitement signalées.
- **Exploration Réseau par OpenVAS** : Ce scanner assure son rôle en identifiant plusieurs vulnérabilités fondamentales créées exprès sur le conteneur, telles que la présence du protocole Telnet transmettant des informations en clair ou les lacunes sur le port FTP et la surface de débogage du réseau cible.
- **Analyse Structurale via Trivy** : Enfin, le scan de composition corrobore parfaitement notre implémentation de logiciels délibérément affaiblis. Le rapport recense exhaustivement l'historique des CVE lié à notre distribution Bionic Beaver obsolète et note formellement toute l'étendue des bibliothèques Python à risque.

Cet alignement entre les vulnérabilités sciemment documentées dans notre matrice de référence et leurs identifications directes par les scanners confirme l'excellence de FileTransferLike en tant que jeu de données qualifié. Les failles ne sont pas seulement théoriques, elles sont tangibles, mesurables par des tiers, et garantissent la solidité du corpus pour l'entraînement de VET.

4 Ingestion des données

4.1 Base de données SQLite

Le module `cve_loader.py` gère la création et l'alimentation de la base de données SQLite [17] qui contient deux tables principales :

Table `cve` (table centrale). Elle stocke toutes les informations techniques d'une vulnérabilité : identifiant CVE, description, scores CVSS et EPSS, vecteur d'attaque, CWE, vendeur, produit, et statut KEV.

Table `server_vulns` (liaison serveur-CVE). Elle trace quelles CVE affectent quels serveurs dans l'infrastructure, permettant une analyse contextuelle.

4.2 Sources de données

L'enrichissement de la base de données s'effectue en quatre phases :

1. **Collecte NVD** [13] : Téléchargement via l'API NVD 2.0, filtré par mots-clés liés aux serveurs HTTP/Web (Apache, Nginx, Flask, Tomcat, etc.). Chaque CVE est parsée pour extraire le CVSS ($\geq v3.1$), le CWE, et les CPE (vendeur/produit).
2. **Enrichissement KEV** [3] : Le catalogue CISA KEV est téléchargé et croisé avec la base. Chaque CVE connue comme activement exploitée est marquée avec `in_kev=1`.
3. **Scores EPSS** [5] : Les scores EPSS (Exploit Prediction Scoring System) sont téléchargés par lots de 30 CVE via l'API FIRST.org, fournissant une probabilité d'exploitation à 30 jours.
4. **Mapping Trivy** : Le rapport XML généré par Trivy [1] est parsé par `trivy_loader.py` pour peupler la table `server_vulns`.

4.3 Indexation vectorielle (RAG)

Pour permettre la recherche sémantique, les descriptions des CVE sont transformées en vecteurs d'embedding et indexées :

Modèle d'embedding. Le modèle `all-MiniLM-L6-v2` [15] (Sentence Transformers) transforme chaque description de CVE en un vecteur dense de 384 dimensions.

Vectorstore FAISS. FAISS (Facebook AI Similarity Search) [8] est utilisé comme base de données vectorielle pour sa rapidité de recherche approchée (ANN). L'index est construit par le module `utils/vectorstore.py`, qui :

1. Récupère toutes les descriptions CVE depuis SQLite,
2. Crée des documents LangChain avec métadonnées (CVSS, EPSS, KEV, vendeur),
3. Indexe via `FAISS.from_documents()`,
4. Sauvegarde l'index localement pour chargement rapide ultérieur.

5 Moteur de priorisation : VMC et XGBoost

5.1 Pourquoi on a besoin d'un moteur de priorisation ?

Quand on travaille dans un SOC, le vrai problème ce n'est pas de trouver les vulnérabilités. Les scanners font ça très bien. Le vrai problème, c'est de savoir lesquelles traiter en premier.

Sur notre serveur vulnérable, on a trouvé 175 vulnérabilités avec les outils de scan (OWASP ZAP, OpenVAS, Trivy). On en a gardé 25 pour les tests. Mais même avec 25 vulnérabilités, si on les trie uniquement par score CVSS, on se retrouve avec beaucoup de « CRITIQUE » alors que certaines ne seront jamais exploitées dans la réalité.

C'est pour ça qu'on a mis en place un moteur de priorisation qui combine plusieurs sources d'information pour donner un score plus réaliste.

5.2 Le framework VMC (Vulnerability Management Chaining)

Principe général. VMC est un arbre de décision. L'idée vient d'un article scientifique de Shimizu et Hashimoto [16] qui montre que combiner CVSS, EPSS et KEV permet de réduire jusqu'à 95% le nombre de vulnérabilités « urgentes » à traiter, tout en gardant une bonne couverture des vraies menaces.

Le principe repose sur quatre questions, posées dans l'ordre suivant :

1. La vulnérabilité est-elle dans le catalogue KEV de la CISA ? Si oui, c'est une menace confirmée avec exploitation active.
2. Sinon, son score EPSS est-il élevé ? Plus le score est haut, plus il y a de chances qu'elle soit exploitée dans les 30 prochains jours.
3. Quel est son impact technique selon le CVSS ?
4. La vulnérabilité permet-elle une exécution de code à distance (RCE) ? Le serveur est-il critique ?

Le calcul du score VMC. Dans notre implémentation (fichier `priority_engine.py`), on calcule un score entre 0 et 100 selon la formule suivante :

$$S_{VMC} = (CVSS \times 4) + (20 \cdot \mathbb{I}_{KEV}) + (EPSS \times 20) + (10 \cdot \mathbb{I}_{RCE}) + 5 \quad (1)$$

Ce score se décompose en trois composantes pondérées :

- **Base (40%)** : le score CVSS multiplié par 4. Comme CVSS varie de 0 à 10, cette composante contribue jusqu'à 40 points et mesure la gravité technique de la faille.
- **Menace (40%)** : 20 points si la CVE figure dans le catalogue KEV (exploitation confirmée dans la nature), plus jusqu'à 20 points selon le score EPSS. Par exemple, un EPSS de 0.5 ajoute 10 points.
- **Contexte (20%)** : 10 points si la description contient des indicateurs de RCE, et 5 points fixes pour la criticité du serveur.

Classification en niveaux de priorité. Une fois le score calculé, chaque vulnérabilité est classifiée dans l'un des quatre niveaux décrits dans le Tableau 3.

TABLE 3. Niveaux de priorité VMC

Score VMC	Niveau	Interprétation
≥ 90	CRITIQUE	Action immédiate requise
≥ 70	HAUTE	Traitement prioritaire
≥ 40	MOYENNE	À planifier
< 40	FAIBLE	Surveillance simple

Pour illustrer l'apport de cette approche par rapport au CVSS seul : une CVE avec CVSS 9.8, présente dans KEV, EPSS à 0.85, et permettant un RCE obtient un score VMC proche de 100 — clairement CRITIQUE. En revanche, une CVE avec CVSS 9.0 mais EPSS à 0.001 et absente du catalogue KEV obtient un score autour de 36 — théoriquement grave, mais concrètement peu risqué.

5.3 La couche Machine Learning : XGBoost

Pourquoi ajouter du Machine Learning ? L'arbre de décision VMC fournit une baseline claire et explicable, mais il a une limite : ses règles sont fixes. Il ne capture pas les interactions complexes entre les variables. Par exemple, il ne détecte pas facilement qu'un vendeur particulier combiné à un type de faille spécifique présente historiquement un profil d'exploitation plus dangereux.

C'est pour cette raison qu'on a ajouté une couche XGBoost dans `data_engine.py`. XGBoost est un algorithme de gradient boosting : au lieu de construire un seul arbre de décision, il en construit des centaines en séquence, chaque arbre corrigeant les erreurs du précédent. Ce mécanisme lui permet d'apprendre des patterns non linéaires que des règles fixes ne peuvent pas capturer.

Génération des données d’entraînement. Un défi classique en apprentissage supervisé est la disponibilité de données labellisées. Dans notre contexte, on ne disposait pas d’exemples annotés par des analystes. La solution adoptée est une **supervision automatique** : on utilise les décisions de l’arbre VMC comme labels d’entraînement. Pour chaque CVE dans la base de données, on applique les règles VMC pour obtenir une priorité (CRITICAL, HIGH, MEDIUM, LOW), et c’est cette priorité qui devient le label fourni à XGBoost.

Cette approche permet à XGBoost de généraliser les règles métier VMC, en capturant des interactions que l’arbre simple ne détecterait pas — par exemple, qu’une vulnérabilité Apache avec une injection SQL est souvent dangereuse même si l’EPSS n’est pas très élevé.

Features utilisées. Le modèle XGBoost s’appuie sur cinq features décrites dans le Tableau 4.

TABLE 4. Features utilisées par le modèle XGBoost

Feature	Type	Description
<code>cvss_score</code>	Numérique (0–10)	Score de sévérité technique
<code>epss_score</code>	Numérique (0–1)	Probabilité d’exploitation à 30 jours
<code>in_kev</code>	Binaire (0/1)	Présence dans le catalogue KEV
<code>vendor_encoded</code>	Catégoriel encodé	Vendeur (Apache=1, Nginx=2, Microsoft=3...)
<code>cwe_encoded</code>	Catégoriel encodé	Type de faille (SQLi=1, XSS=2, RCE=3...)

Ces features combinent des informations quantitatives (les scores) avec des informations qualitatives (vendeur, type de faille). Ce choix s’inspire des résultats du modèle EPSS v3 [7], qui montre que les signaux les plus prédictifs de l’exploitation incluent justement les mots-clés dans la description, le vendeur, et la présence dans des listes d’exploitation connues.

Processus d’entraînement. L’entraînement est implémenté dans la méthode `train_xgboost()` de la classe `VMCXGBoostEngine` et se déroule en six étapes :

1. Chargement de toutes les CVE depuis la base de données SQLite via le `PriorityEngine`.
2. Application des règles VMC pour générer les labels de classification.
3. Encodage des variables catégorielles avec un `LabelEncoder`.
4. Division des données : 80% pour l’entraînement, 20% pour le test.
5. Entraînement XGBoost en classification multi-classes (`objective='multi:softprob', max_depth=6, learning_rate=0.1, n_estimators=100`).
6. Sauvegarde du modèle et des encodeurs avec `joblib`.

Prédiction et mécanisme de fallback. Lors de l’inférence, le modèle calcule une probabilité pour chacune des quatre classes et retourne la classe avec la probabilité la plus élevée, accompagnée d’un score de confiance. Un mécanisme de fallback est intégré : **si la confiance est inférieure à 80%**, le système affiche également l’avis de l’arbre VMC. L’analyste peut ainsi voir les deux opinions et trancher.

Ce design respecte le principe du *Human In The Loop* [12] : le modèle propose, mais le jugement humain reste central.

5.4 L’orchestration : le VMC Tool

Le fichier `vmc_tool.py` joue le rôle de coordonnateur. C’est lui qui est appelé par l’agent `LangChain` lorsque l’analyste pose une question sur les vulnérabilités. Son fonctionnement suit quatre étapes :

1. Appel au `PriorityEngine` pour récupérer la liste des CVE avec leurs scores VMC.

2. Pour chaque CVE, demande d'une prédiction XGBoost à la couche ML.
 3. Construction d'un contexte enrichi pour le LLM, avec variables structurées (nombre de critiques, CVE la plus dangereuse, EPSS moyen).
 4. Formatage en texte clair pour Llama 3.1. Exemple :
1. CVE-2021-44228 [CRITIQUE] (VMC: 95.0)
 EPSS: 0.9750, KEV: OUI
 Apache Log4j remote code execution...

Ce format compact permet au LLM de comprendre rapidement la situation et de formuler une réponse justifiée pour l'analyste.

5.5 Avantages de l'approche hybride

La combinaison VMC et XGBoost apporte plusieurs avantages concrets :

- **Transparence** : l'arbre VMC est toujours explicable. On peut justifier chaque décision par ses composantes (KEV, EPSS, CVSS), ce qui est essentiel pour la confiance de l'analyste.
- **Flexibilité** : XGBoost capture des patterns que les règles fixes ne voient pas, comme des combinaisons de vendeur et type de faille historiquement plus exploitées.
- **Fiabilité** : le mécanisme de fallback garantit qu'on a toujours une réponse exploitable. En cas de faible confiance, l'analyste dispose des deux avis.
- **Cohérence** : comme XGBoost est entraîné sur les décisions VMC, les deux systèmes ne se contredisent pas fondamentalement. XGBoost raffine les règles VMC sans les remplacer.

5.6 Résultats observés sur notre dataset

Sur les 25 vulnérabilités de test issues de notre serveur Docker, le moteur a produit une distribution cohérente avec les attentes. Les CVE ayant obtenu les scores les plus élevés sont :

- **CVE-2004-2687** (DistCC RCE) : présente dans KEV, EPSS élevé, RCE confirmé → score VMC $\approx$ 95.
- **CVE-2017-0144** (EternalBlue/SMBv1) : présente dans KEV, associée à des campagnes de ransomware actives → niveau CRITIQUE.
- **CVE-2014-6271** (Shellshock) : présente dans KEV, code d'exploitation très répandu → niveau CRITIQUE.

À l'opposé, des vulnérabilités avec un CVSS élevé mais sans exploitation confirmée (absentes de KEV, EPSS faible) ont été classées MOYENNE ou FAIBLE. Ce résultat est cohérent avec les conclusions de [16] : l'approche permet de réduire de 95% le volume de vulnérabilités « urgentes » tout en maintenant une couverture de 85% des vraies menaces, aidant ainsi l'analyste à concentrer son attention sur les risques réels.

6 Système RAG et Agent LLM

6.1 Architecture RAG

Le système RAG (Retrieval-Augmented Generation) [10] constitue le cœur intelligent de VET. Il permet au LLM de répondre avec précision en s'appuyant sur des données factuelles extraites de la base de connaissances, plutôt que de s'appuyer uniquement sur ses connaissances pré-entraînées.

Modèle LLM. VET utilise le modèle **Llama 3.1 8B** [11] de Meta, exécuté localement via `llama.cpp` [6] au format quantifié GGUF (Q5_K_M). Ce choix garantit :

- **Confidentialité** : aucune donnée ne quitte l’infrastructure (pas d’API cloud),
- **Performance** : temps de réponse acceptable avec accélération GPU,
- **Qualité** : le modèle 8B offre un bon équilibre entre taille et capacité d’analyse complexe.

La configuration du LLM est centralisée dans `config.py` : température basse (0.1) pour la reproductibilité, contexte de 8192 tokens, et pénalité de répétition de 1.2.

6.2 Outils de l’agent

L’agent `SecurityAssistant` orchestre cinq outils spécialisés, chacun implémentant l’interface `StructuredTool` de LangChain [9] :

Outil	Module	Fonction
CVE RAG Search	<code>retriever_tool.py</code>	Recherche hybride SQL + vectorielle
VMC Priorities	<code>vmc_tool.py</code>	Classement VMC + scores XGBoost
Risk Calculator	<code>risk_calculator.py</code>	Score de risque dynamique (EPSS + logs)
SOC Recommender	<code>soc_recommender.py</code>	Recommandations contextuelles SOC
SOC Summary	<code>soc_summary.py</code>	Génération de rapports PDF automatiques

TABLE 5. Outils de l’agent `SecurityAssistant`.

6.3 Recherche hybride

Le `ServerHybridRetriever` implémente une stratégie de recherche en trois étapes :

1. **Requête SQL** : Récupération des CVE associées au serveur cible depuis `server_vulns`,
2. **Filtrage vectoriel** : Si des CVE serveur sont trouvées, recherche sémantique FAISS filtrée sur ces CVE (top 100 résultats, filtrage Python, rétention des top 8),
3. **Fallback global** : Si aucun résultat n’est trouvé pour le serveur, recherche vectorielle globale (top 8).

Cette approche garantit la pertinence contextuelle : les résultats sont toujours liés au serveur analysé, tout en permettant un fallback vers la base complète si nécessaire.

6.4 Classification d’intention

Le système utilise un mécanisme de classification d’intention en deux couches :

Override par règles. Avant tout appel au LLM, des règles déterministes détectent certaines intentions via des expressions régulières :

- Détection de pattern CVE (`CVE-\d{4}-\d+`) → intention `analyze_cve`,
- Mots-clés de logs (`logs`, `traces`, `exploitation`) → intention `logs`.

Classification LLM. Pour les cas non couverts par les règles, le LLM classe l’intention en retournant un JSON structuré : `{security, intent, confidence}`.

6.5 Génération structurée (2-Pass LLM)

VET utilise une architecture à deux passes :

Pass 1 — Analyse structurée. Le LLM génère une réponse conforme à un schéma Pydantic [14]. Par exemple, pour l'intention `list_vulns`, le schéma `CVEClassement` impose :

- `total_vulns` : nombre total de vulnérabilités,
- `top_recommendation` : CVE la plus dangereuse,
- `prioritized_list` : liste structurée des top 3 CVE avec EPSS, KEV, description.

Pass 2 — Reformulation naturelle. La sortie structurée est ensuite reformulée par le LLM en dialogue naturel, adapté au rôle de l'interlocuteur (analyste SOC).

6.6 Calcul de risque dynamique

Le `risk_calculator.py` implémente un calcul de risque combinant EPSS et analyse de logs :

$$P(\text{exploit}) = P(\text{EPSS}) + P(\text{logs}) - P(\text{EPSS}) \times P(\text{logs}) \quad (2)$$

Cette formule correspond à la probabilité de l'union de deux événements ($P(A \cup B)$). Le facteur log est fixé à 0.9 en cas d'exploitation détectée et 0.4 en cas d'activité suspecte.

6.7 Analyse de logs

Le module `SecurityLogAnalyzer` dans `utils/log_analyzer.py` analyse les logs de sécurité en temps réel pour détecter des patterns d'attaque :

- **Injection SQL** : Détection de patterns comme `UNION SELECT`, `1=1`, `OR 1=1`,
- **XSS** : Recherche de balises `<script>` et encodages malveillants,
- **Traversée de répertoire** : Détection de `../` et variantes encodées,
- **Injection de commandes** : Patterns de pipe (`|`), point-virgule, backticks,
- **Force brute** : Comptage de tentatives échouées par IP source.

L'analyse produit un rapport structuré incluant : le statut d'exploitation, le nombre de tentatives suspectes, les IP sources, et les types d'attaque détectés.

6.8 Interface utilisateur Streamlit

L'interface est développée avec Streamlit [18] et offre :

- Un **chat conversationnel** avec mémoire (historique de conversation),
- Un **tableau de bord** de distribution des risques (VMC),
- La **génération de rapports PDF** téléchargeables,
- Un sélecteur de serveur pour l'analyse contextuelle.

7 Travaux connexes

7.1 Systèmes de gestion des vulnérabilités

Les outils commerciaux de gestion des vulnérabilités (Qualys, Tenable Nessus, Rapid7 InsightVM) fournissent des capacités de scan et de priorisation. Cependant, leur approche de priorisation repose principalement sur le score CVSS, sans intégrer de manière native :

- La corrélation avec les logs d'exploitation réels,
- L'analyse conversationnelle en langage naturel,
- Le Machine Learning pour la priorisation adaptative.

VET se différencie en combinant ces trois aspects dans un assistant conversationnel unifié.

7.2 RAG pour la cybersécurité

Le paradigme RAG [10], initialement proposé pour les tâches NLP knowledge-intensive, a été appliqué dans plusieurs contextes de cybersécurité, notamment pour l’analyse de rapports de menaces (threat intelligence) et la classification de malwares. VET adapte ce paradigme au contexte SOC en utilisant une base de connaissances structurée (CVE + EPSS + KEV) plutôt que des documents texte bruts.

7.3 ML pour la priorisation de vulnérabilités

L’utilisation du Machine Learning pour la priorisation des vulnérabilités est un domaine de recherche actif. Les travaux récents explorent les modèles de gradient boosting (XGBoost, LightGBM) pour prédire l’exploitabilité des CVE à partir de features textuelles et structurées. VET adopte XGBoost [2] avec une approche originale : l’entraînement supervisé automatique à partir des décisions de l’arbre VMC, créant ainsi un système où le ML apprend et généralise les règles métier.

7.4 Comparaison des approches

Critère	VET	Qualys	Nessus	InsightVM
Priorisation CVSS	✓	✓	✓	✓
Intégration EPSS	✓	Partiel	Non	Partiel
Catalogue KEV	✓	✓	Non	✓
Corrélation logs	✓	Non	Non	Partiel
ML (XGBoost)	✓	Non	Non	Non
Interface NLP	✓	Non	Non	Non
LLM local	✓	Non	Non	Non
Open-source	✓	Non	Non	Non

TABLE 6. Comparaison de VET avec les solutions commerciales.

Les principaux avantages de VET sont l’intégration d’un LLM local pour le dialogue en langage naturel et la combinaison de priorisation ML avec corrélation de logs. Les limitations incluent le périmètre de scan (dépendant d’outils tiers) et la nécessité de ressources GPU pour des performances optimales.

8 Conclusion et perspectives

8.1 Bilan

Le projet VET démontre la faisabilité d’un assistant SOC intelligent, capable d’analyser et de prioriser les alertes de sécurité en combinant un grand modèle de langage localisé, une architecture RAG et un moteur de priorisation hybride. Les principaux résultats de cette implémentation sont :

- **Un pipeline RAG contextuel de bout en bout** : Le système couvre tout le processus, de l’ingestion des bases de vulnérabilités NVD jusqu’au dialogue de réponse. L’utilisation d’une recherche hybride, alliant ciblage SQL précis puis filtrage vectoriel via FAISS, permet d’assurer que l’IA répond toujours en se basant sur la réalité technique du serveur ciblé.
- **Un moteur de priorisation hybride explicable** : Nous avons combiné une approche déterministe sous forme d’arbre de décision, le Vulnerability Metric Classifier exploitant le CVSS, l’EPSS et le KEV, à un modèle XGBoost. XGBoost apprend des décisions de l’arbre pour les généraliser, ce qui nous a permis de filtrer drastiquement les urgences sans compromettre la détection des menaces avérées.

- **Une corrélation Logs et Vulnérabilités dynamique** : L'innovation clé réside dans la pondération des faits réels. Une vulnérabilité critique théorique devient une alerte absolue et son score bondit si notre analyseur détecte des traces d'attaques correspondantes dans les logs, par exemple des motifs d'injection SQL sur une application vulnérable.
- **Une génération structurée en architecture double passe** : VET contraint d'abord le LLM avec des schémas Pydantic pour s'assurer que les données extraites sont fiables et au bon format, comme les listes de CVE et les indicateurs de risque. Ensuite, une deuxième passe LLM reformule ces résultats bruts en un langage naturel et professionnel.
- **Un outil respectueux de la confidentialité** : En exploitant le modèle Llama 3.1 8B exécuté localement, on garantit qu'aucune donnée de production sensible de l'entreprise, tels que les journaux, adresses IP ou infrastructures, n'est envoyée vers des serveurs tiers ou des API cloud grand public.

8.2 Défis et obstacles rencontrés

La conception de VET s'est heurtée à plusieurs complexités techniques liées à la nature même de l'IA générative :

- **Les hallucinations du LLM et le formatage** : Avec un modèle de la taille de Llama 8B, nous avons des dérives où le LLM inventait des données sur les CVE ou cassait les structures JSON attendues. L'implémentation de la méthode à double passe et de consignes strictes dans les prompts, comme l'interdiction formelle d'inventer des données, a considérablement fiabilisé le système.
- **La classification des intentions de l'utilisateur** : L'outil devant choisir la bonne procédure entre lire les logs, interroger le NVD ou faire un résumé, laisser le LLM deviner l'intention cible générait des erreurs de routage. Nous l'avons pallié par un filtrage basique mais robuste en amont : des expressions régulières qui forcent le choix de l'outil approprié si la demande est évidente avec des mots-clés précis liés aux CVE.
- **La fluidité du dialogue face à un format structuré** : Réussir à garder un dialogue naturel avec l'utilisateur tout en imposant au système un format d'extraction de données très rigide a été un vrai défi de développement. C'est précisément pour cela que l'architecture technique à double passe est devenue incontournable, elle sépare l'extraction pure de la formulation conversationnelle.

8.3 Perspectives d'évolution

Plusieurs améliorations pourraient être envisagées pour rapprocher ce prototype des standards de la production :

- **Le Human-in-the-Loop et l'amélioration continue** : L'idée est de permettre à un analyste senior SOC de faire des retours explicites sur les réponses de l'IA. Si le modèle propose une recommandation, l'analyste devrait pouvoir la corriger ou lui indiquer de nouveaux éléments. L'IA pourrait ainsi réviser son analyse en direct et intégrer cet avis critique pour affiner son contexte de réponse lors des sollicitations futures.
- **Orchestration améliorée et corrélation croisée via LangGraph** : Le passage à un orchestrateur plus puissant permettrait au système d'utiliser une chaîne d'informations plus intelligemment. L'IA pourrait croiser par elle-même les différents outils pour assurer une bien meilleure mise en contexte, par exemple en cherchant une CVE de manière isolée pour ensuite aller chercher les adresses IP correspondantes dans les logs, garantissant ainsi une corrélation redoutablement plus riche lors d'incidents complexes.
- **Adoption de modèles spécialisés** : Le modèle de test Llama 3.1 8B est généraliste. Le remplacer par un modèle de plus grande dimension ou, idéalement, un modèle ayant subi un apprentissage spécifique sur la rhétorique et les documents propres à la cybersécurité ferait bondir la qualité rédactionnelle et d'analyse.

- **Intégration SIEM en temps réel** : Notre analyseur de logs fonctionnant actuellement avec des exports statiques textuels, la suite logique serait de connecter directement VET à des solutions du marché comme Elastic ou Splunk pour le traitement automatique du flux en temps réel.

Références

1. Aqua Security : Trivy : The All-in-One Security Scanner. <https://trivy.dev/>, consulté le 27 février 2026
2. Chen, T., Guestrin, C. : Xgboost : A scalable tree boosting system. In : Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16). pp. 785–794. ACM (2016). <https://doi.org/10.1145/2939672.2939785>
3. Cybersecurity and Infrastructure Security Agency (CISA) : Known Exploited Vulnerabilities (KEV) Catalog. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>, consulté le 27 février 2026
4. FIRST.Org, Inc. : Common Vulnerability Scoring System v3.1 : Specification Document. <https://www.first.org/cvss/v3-1/specification-document>, consulté le 27 février 2026
5. FIRST.Org, Inc. : Exploit Prediction Scoring System (EPSS). <https://www.first.org/epss>, consulté le 27 février 2026
6. ggml-org : llama.cpp : LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, consulté le 27 février 2026
7. Jacobs, J., Romanosky, S., Suci, O., Edwards, B., Sarabi, A. : Enhancing vulnerability prioritization : Data-driven exploit predictions with community-driven insights. arXiv preprint arXiv :2302.14172 (2023), <https://arxiv.org/abs/2302.14172>
8. Johnson, J., Douze, M., Jégou, H. : Billion-scale similarity search with gpus. IEEE Transactions on Big Data **7**(3), 535–547 (2019)
9. LangChain : LangChain Documentation. <https://docs.langchain.com/>, consulté le 27 février 2026
10. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D. : Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv :2005.11401 (2020), <https://arxiv.org/abs/2005.11401>, accepté à NeurIPS 2020
11. Meta AI : Llama 3.1 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md (2024), consulté le 27 février 2026
12. National Academies of Sciences, Engineering, and Medicine : Large Language Models and Cybersecurity : Proceedings of a Workshop—in Brief. The National Academies Press, Washington, DC (2024). <https://doi.org/10.17226/27776>, <https://www.nationalacademies.org/read/27776>, référence utilisée pour justifier l'approche *Human-in-the-Loop*.
13. National Institute of Standards and Technology (NIST) : National Vulnerability Database (NVD) – 2.0 APIs / Data Sources. <https://nvd.nist.gov/developers/data-sources>, consulté le 27 février 2026
14. Pydantic Team : Announcement : Pydantic V2 Release. <https://pydantic.dev/articles/pydantic-v2-final> (2023), publié le 30 juin 2023 ; consulté le 27 février 2026
15. Reimers, N., Gurevych, I. : Sentence-bert : Sentence embeddings using siamese bert-networks. arXiv preprint arXiv :1908.10084 (2019), <https://arxiv.org/abs/1908.10084>
16. Shimizu, N., Hashimoto, M. : Vulnerability management chaining : An integrated framework for efficient cybersecurity risk prioritization. arXiv preprint arXiv :2506.01220 (2025), <https://arxiv.org/abs/2506.01220>, clé conservée (shimizu2023vmc) pour correspondre au L^AT_EX ; publication 2025.
17. SQLite Consortium : SQLite Documentation. <https://www.sqlite.org/docs.html>, consulté le 27 février 2026
18. Streamlit Inc. : Streamlit Documentation. <https://docs.streamlit.io/>, consulté le 27 février 2026